

Using Conda to Create and Manage Environments

What is Conda?

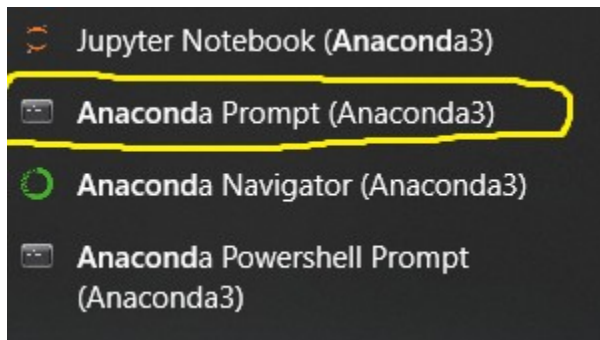
- Conda is an open-source package management and environment management system.
- Works with Windows, macOS, and Linux.
- Can manage dependencies and create isolated environments.
- Supports multiple programming languages (Python, R, etc.).

Benefits of Using Conda Environments

- Isolate project dependencies to avoid conflicts.
- Create reproducible environments for sharing with others.
- Simplify package management.
- Easily switch between different environments.

Setting Up Conda

- **Install Conda** via Anaconda or Miniconda.
- **Verify** the installation with: `conda --version`
- **Update** Conda to the latest version with `conda update conda`



Creating a New Environment

- Create an environment with a specific Python version:
`conda create --name myenv python=4.14`
- Activate the environment:
`conda activate myenv`
- Deactivate the environment:
`conda deactivate`

Managing Environments

- Listing all environments:

```
conda env list
```

- Removing an environment:

```
conda remove --name myenv --all
```

- Cloning an environment:

```
conda create --name newenv --clone  
myenv
```

Installing and Managing Packages

- Installing a package:

`conda install numpy`

- Updating a package:

`conda update numpy`

- Removing a package:

`conda remove numpy`

Exporting and Sharing Environments

- Exporting an environment to a [YAML](#) file:

`conda env export > environment.yml`

- Creating an environment from a YAML file:

`conda env create -f environment.yml`

- Regularly update your environments and packages.
- Use separate environments for each project.
- Document your environment setup with an `environment.yml` file.
- Avoid using the base environment for projects.
 - **To prevent dependency conflict:** The base environment is the default environment that comes with Conda installation
 - **Reproducibility:** The base environment is not isolated, so changes made in one project can affect other projects if they share the same environment.
 - **Environment Isolation:** The base environment is a shared environment where all packages are installed globally.
 - **Ease of Maintenance:** Managing packages in the base environment can become cumbersome over time as more projects are developed.
 - **Safety of the Conda Installation:** The base environment contains essential packages that Conda relies on.

UTILISING CLOUD CODING ENVIRONMENTS WITH GOOGLE COLAB

Cloud coding environments i.e. Google Colab

- Cloud coding environments allow you to write, run, and share code through a web-based interface.
- Benefits include no local setup, access to powerful computing resources, and easy collaboration.
- Examples: Google Colab, Jupyter Notebooks on the cloud, AWS SageMaker, Microsoft Azure Notebooks.

- Google Colab (Collaboratory) is a free cloud-based Jupyter notebook environment.
- Hosted by Google and integrated with Google Drive.
- Supports Python and popular libraries such as TensorFlow, PyTorch, Keras, and more.
- Provides free access to graphics processing units (GPUs) and Tensor Processing units (TPUs) for accelerated computing.
- Google Colab primarily supports Python, but it can also be configured to run R.

Colab: Features & Set Up and Access

- No setup required: Start coding immediately in a web browser.
- Integrated with Google Drive: Save and access your notebooks easily.
- Hardware accelerators: Free access to GPUs and TPUs for faster computations.
- Collaboration: Share notebooks with others and work together in real-time.
- Extensive library support: Pre-installed libraries for machine learning, data science, and more.
- Access Google Colab at <https://colab.research.google.com/>
- Sign in with a Google account.
- Create a new notebook by clicking on “New Notebook.”
- Notebooks are automatically saved to your Google Drive.

Writing and Running Code in Google Colab

- **Code cells:** Write and execute Python code.
- **Markdown cells:** Add text, images, and formatted content.
- **Running cells:** Click the “Run” button or press Shift + Enter. (or “Cmd + Enter” on macOS)
- **Accessing files:** Upload files or connect to Google Drive.
- **Installing libraries:** Use ***!pip install*** to add additional libraries.

Collaborating on Projects

- **Sharing notebooks:** Share via Google Drive or a direct link.
- **Real-time collaboration:** Multiple users can edit the same notebook simultaneously.
- **Version control:** View and revert to previous versions of the notebook.
- **Commenting:** Add comments to specific cells for discussion.

Best Practices for Using Google Colab

- Regularly save your work: Notebooks are automatically saved, but it's good to save checkpoints.
- Manage resources: Monitor and manage your usage of GPU/TPU resources.
- Use environment variables: Store sensitive information securely.
- Install only necessary libraries: Keep the environment clean and minimize install times.